

Day 5: AWS Compute Overview

EC2, Auto Scaling & Elastic Load Balancing (ELB)

Welcome back, everyone! Today we move from theory into the actual engines that power cloud applications. By the end of this session, you'll know how to launch, size, scale, and load-balance virtual servers on AWS — skills that appear in nearly every cloud support technician job posting.

ITNW 1009 — DAY 5

AWS COMPUTE



Today's Agenda

Here's the roadmap for our 3.5-hour session. We'll build up our understanding layer by layer — from a single virtual machine all the way to a fully scaled, load-balanced web application.



Amazon EC2 Fundamentals

Anatomy, instance families, and sizing logic



Pricing Models

On-Demand, Reserved, Spot, and Savings Plans



AMIs & Launch Templates

The automation blueprint for rapid deployment



Auto Scaling Groups (ASG)

Horizontal scalability and dynamic capacity



Elastic Load Balancing (ELB)

Distributing traffic for high availability



Chapter 1

From Physical Servers to the Cloud

Before AWS, building infrastructure meant writing purchase orders, waiting weeks for hardware delivery, and praying you ordered the right size. Let's explore how EC2 changed everything.



Welcome to AWS Compute Architecture

The Old Way 🖥️

Buy physical servers. Wait weeks. Over-provision to avoid crashes. Under-provision and your app goes down. Waste thousands of dollars either way – with no flexibility.

The AWS Way

Amazon EC2 – Elastic Compute Cloud – lets you spin up a fully functional, virtualized operating system environment in under two minutes. Pay only for what you use. Expand or shrink with a few clicks. No data center required.



Career Tip: Every entry-level cloud technician job description requires hands-on EC2 experience. Today you'll build those skills from the ground up.

What Does "Elastic Compute Cloud" Actually Mean?

Cloud

Your virtual machine lives on AWS infrastructure – no physical hardware to manage, rack, or cable.

Compute

It's a real processing environment – CPU, RAM, networking – that runs your operating system and applications.

Elastic

It stretches out or shrinks back based on your actual needs. No waste, no over-provisioning, no wasted budget.

The "C2" simply reflects that the word **Cloud** and the word **Compute** both start with C – giving us **EC2**. Simple once you know it!

The Anatomy of an EC2 Instance

An EC2 instance is a **virtual machine**. AWS takes massive physical rack servers inside their data centers and runs a **hypervisor** — software that slices the hardware into smaller, isolated virtual computers. Here's what lives inside each one:



vCPU

Virtual Central Processing Units. You choose exactly how many processing cores your application needs.



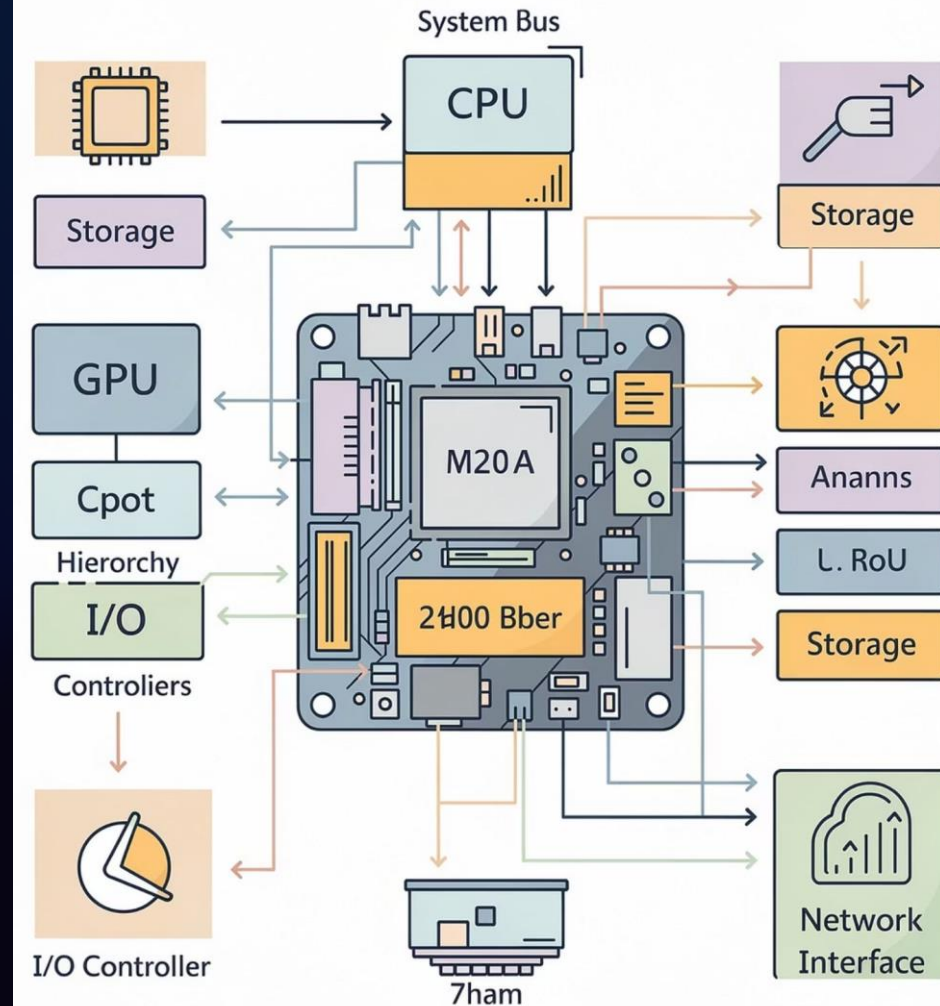
Memory (RAM)

Random-access memory for running active processes. Selected alongside vCPU based on your instance type.



EBS Volume

Your persistent virtual hard drive. Critically, it is **separate** from the compute layer — your data survives an instance restart.



The Key Insight: Compute vs. Storage Are Separated

This trips up many beginners — and it's a concept that appears on the AWS Cloud Practitioner exam. Your **EBS (Elastic Block Store)** volume is attached to your EC2 instance over an ultra-fast internal network, but it is **not soldered into the compute layer**.

Think of it like a desktop tower where the hard drive plugs in via a cable. If you turn the computer off, unplug it, or swap the motherboard — **your files on the drive remain completely intact**.

- ✓ If you resize or stop your EC2 instance, your EBS data is **NOT deleted**. Compute and storage are independent components.

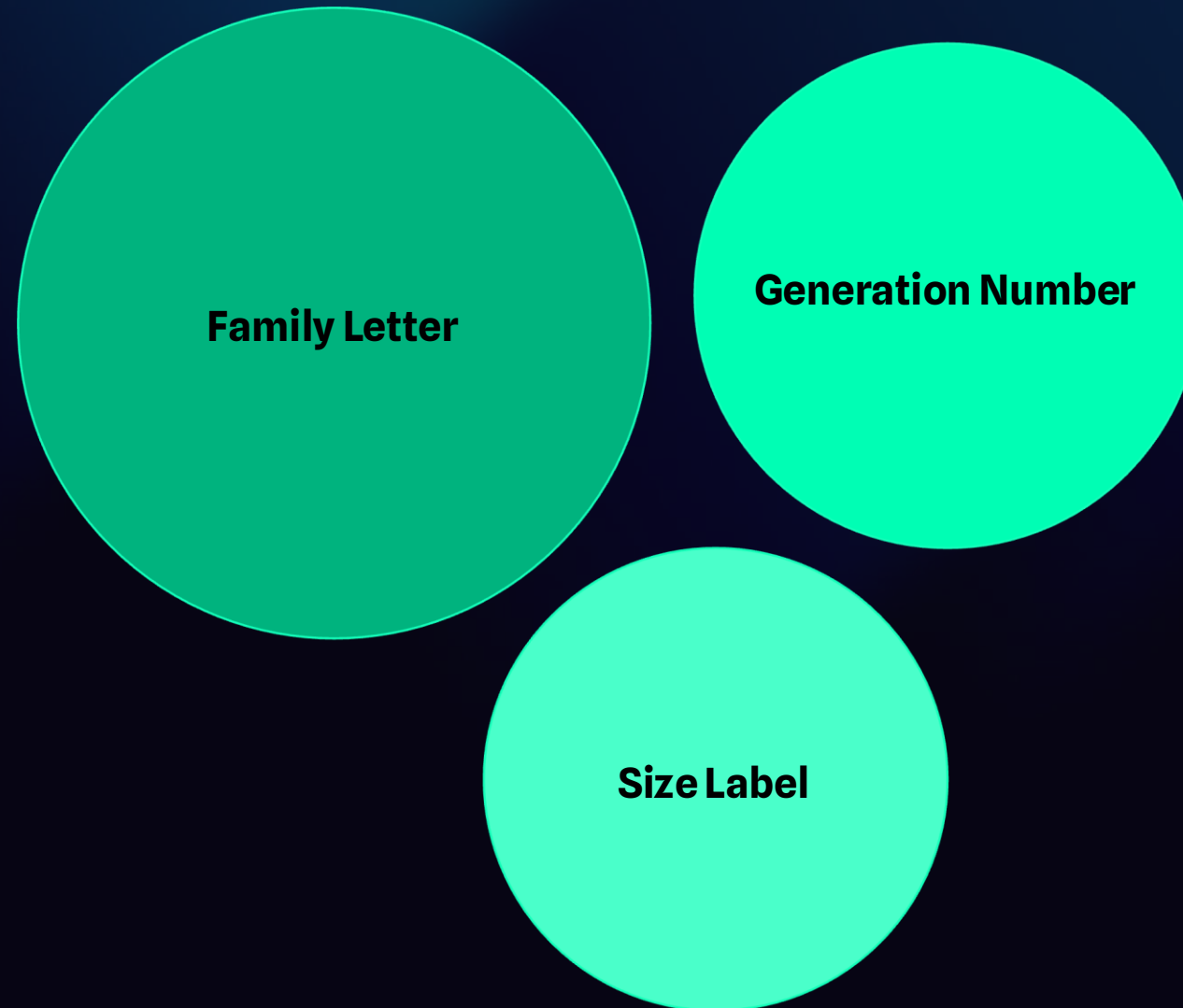
Quick Check

If you change your EC2 instance from a **t3.large** to a **t3.xlarge** because your app needs more RAM — what happens to the files stored on your attached EBS volume?

Answer: They stay completely safe. The storage is separate from the compute configuration.

Decoding the EC2 Instance Naming System

When you open the AWS console, you'll see hundreds of instance names like `t3.micro`, `c6i.xlarge`, and `r6g.large`. Don't panic — there's a clear system behind the naming.



The **family letter** tells you the optimization type. The **generation number** tells you how recent the hardware is (higher = newer). The **size label** works like t-shirt sizing — micro, small, medium, large, xlarge — doubling resources at each step.

EC2 Instance Family Reference Guide

Instance Family	Optimization Type	Real-World Use Case
T3, M5 (General Purpose)	Balanced CPU, RAM, Network	Web servers, dev environments, small business apps
C6i (Compute Optimized)	High vCPU-to-RAM ratio	Data analytics, media encoding, log processing
R6g (Memory Optimized)	High RAM-to-CPU ratio	High-speed databases, real-time caching systems
G4 (Accelerated Computing)	GPU acceleration	Machine learning training, video rendering

 **Memory Trick:** **C** = CPU horsepower. **R** = RAM powerhouse. **M/T** = Middle-of-the-road, balanced Swiss Army knife.

Choosing the Right Instance Family

Real-world scenario practice — your future employer will ask these questions during an interview!

Scenario A: Company Website

A small business needs a web server for their 5-page marketing site and a developer testing environment. **Best choice: T3 or M5 (General Purpose)** — balanced resources at an affordable cost.

Scenario B: Analytics Pipeline

A retail company runs nightly reports crunching millions of sales transaction rows. **Best choice: C6i (Compute Optimized)** — maximum CPU muscle for number-crunching workloads.

Scenario C: Enterprise Database

A healthcare system runs a patient records database that crashes when RAM runs out. **Best choice: R6g (Memory Optimized)** — massive RAM keeps millions of records in ultra-fast memory.

Chapter 2

Smart Cloud Cost Management

Knowing how to keep the AWS bill low is just as valuable as knowing how to build infrastructure. These four pricing models are your toolkit.



The Four EC2 Pricing Models

AWS gives you four ways to pay for compute power. Choosing the right one for the right workload is a core cloud technician skill – and a common exam topic.

On-Demand

Pay by the second. Maximum flexibility, highest baseline rate. Ideal for unpredictable or short-term workloads.

Reserved / Savings Plans

Commit to 1 or 3 years. Save **up to 72%** vs. On-Demand. Best for steady, always-on production systems.

Spot Instances

Bid on spare AWS capacity. Save **up to 90%**. Can be terminated with a 2-minute warning. Safe only for interruptible workloads.

Dedicated Hosts

A physical server exclusively for your use. Required for certain compliance or licensing requirements. Highest cost.

On-Demand: Maximum Flexibility

How It Works

You pay per second (Linux) or per hour (Windows) for exactly the compute time you consume. No contracts. No commitments. No minimums.

Turn it on. Run it. Turn it off. Pay only for those exact minutes.

When to Use It

- New applications with unpredictable traffic patterns
- Short-term testing and development environments
- Workloads that can't be interrupted but run only occasionally
- Disaster recovery standby instances you hope to never need

Analogy: Renting a car by the hour — flexible, but more expensive per mile than a monthly lease.

Reserved Instances & Savings Plans: The Smart Commitment

Reserved Instances

Commit to a specific instance type, region, and OS for **1 or 3 years**. Save up to **72%** vs. On-Demand rates. Best for core production systems that run 24/7/365 — like a company payroll system or authentication server.

Savings Plans

More flexible than Reserved Instances. Commit to a consistent amount of **compute usage in dollars per hour** for 1 or 3 years, rather than locking into a specific instance type. The discount applies automatically across EC2, Lambda, and Fargate.

📌 **Analogy:** Signing a long-term apartment lease. You get a significantly lower monthly rate because the landlord (AWS) values your guaranteed commitment.



Spot Instances: Extreme Savings, With a Catch

The Opportunity 💰

AWS data centers always have pools of unused server capacity sitting idle. To reduce waste, AWS auctions this spare compute power at discounts of **up to 90% off** On-Demand prices.

A Spot instance might cost you **\$0.05/hour** instead of **\$0.50/hour** for the same hardware.

The Risk

If a full-price On-Demand customer needs that hardware, AWS will **terminate your Spot instance with only a 2-minute warning**. Your workload stops — period.

Safe uses: Batch image processing, big data analysis, background rendering jobs — tasks that can pause and safely restart.

Unsafe uses: Customer-facing checkout flows, live databases, authentication systems.

Pricing Model Decision Guide

Use this framework to match the right pricing model to any workload scenario — this is exactly the kind of decision you'll make in a cloud support role.

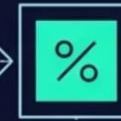
ON-DEMAND

Flexible, short-term,
unpredictable workloads.



RESERVED / SAVINGS PLANS

Steady 24/7 workloads, 1-3
yr commitment, 72% savings.



SPOT INSTANCES

Interruptible batch jobs,
background tasks,
90% savings.



DEDICATED HOSTS

Compliance requirements,
licensed software,
physical isolation.



Pricing Model: Vocational Quiz

Match each real-world business scenario to the most cost-efficient pricing model. These scenarios come directly from AWS certification exam practice questions.

1

Scenario 1

Your company's core HR software runs 24 hours a day, 7 days a week, and has done so for 3 years with no signs of changing. **Answer: Reserved Instances** – commit and save up to 72%.

2

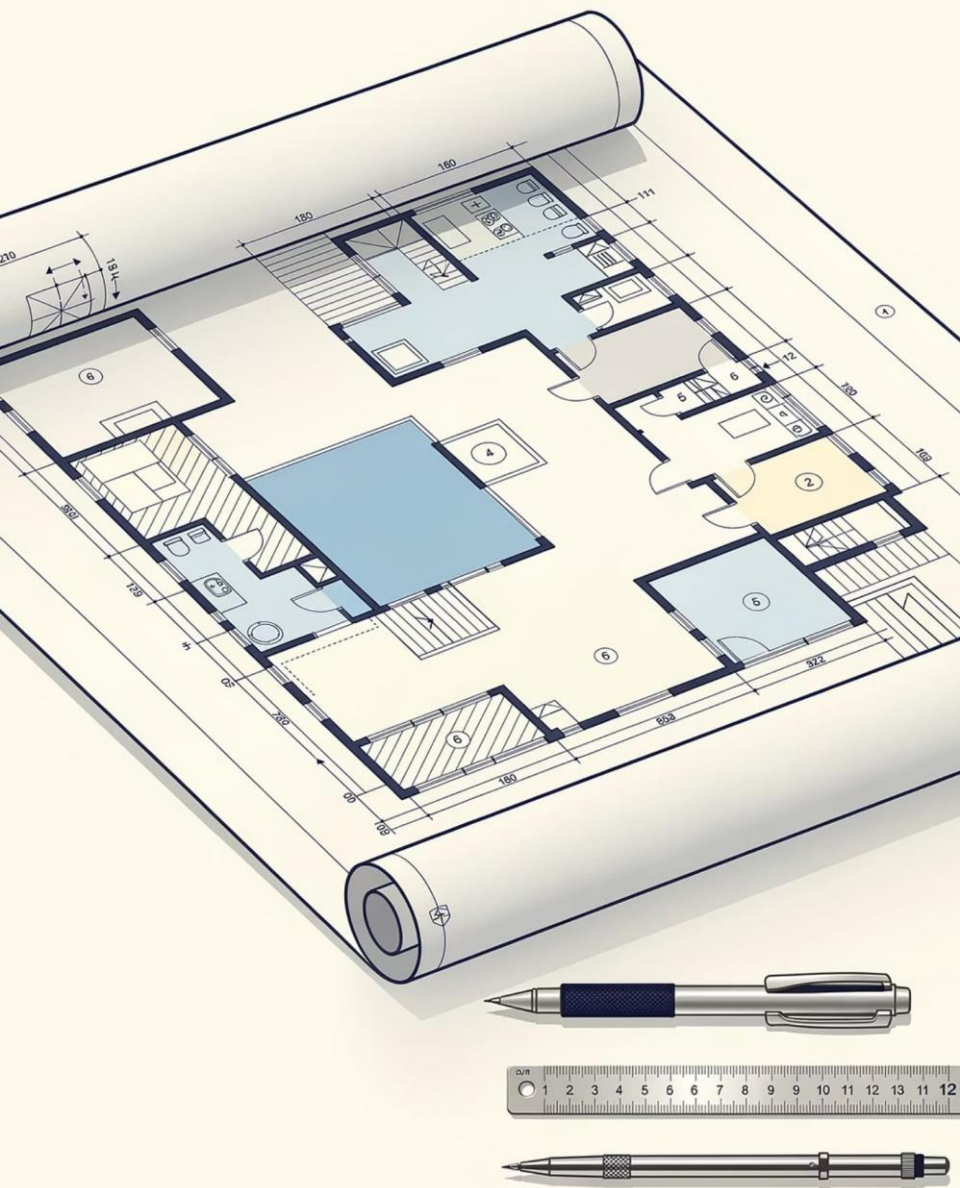
Scenario 2

A startup needs to test a new app for 3 days but doesn't know if it'll succeed. **Answer: On-Demand** – no commitment, pay only for the test window.

3

Scenario 3

A media company renders thousands of video thumbnails overnight on weekends. Jobs can restart if interrupted. **Answer: Spot Instances** – save up to 90% on batch-safe workloads.



Chapter 3

AMIs & Launch Templates

Before you can automate scaling, you need a perfect blueprint. Amazon Machine Images and Launch Templates are the foundation of infrastructure automation.

Amazon Machine Image (AMI): The Master Template

What Is an AMI?

An **Amazon Machine Image (AMI)** is a frozen snapshot of a virtual machine's entire configuration — the operating system, installed software packages, security patches, application code, and custom settings — all baked into a single reusable image file.

Think of an AMI as a **master recipe**. Once you've perfected your server setup, you freeze that recipe so you can recreate identical servers instantly — without manually configuring each one from scratch.

AWS provides pre-built public AMIs for common setups like **Amazon Linux 2, Ubuntu, Windows Server**, and many more. You can also create your own custom AMI from a running instance you've already configured.

AMI Contains:

- Operating System (Linux, Windows, etc.)
- Pre-installed software packages
- Security patches applied
- Custom application code
- Root volume storage configuration

Launch Templates: The Complete Blueprint

An AMI handles the *software* side of a server. But AWS also needs to know the *infrastructure* settings. That's where a **Launch Template** comes in — it wraps everything into one saved master file.



What a Launch Template Stores

AMI ID, instance type (e.g. t3.medium), key pair for SSH access, VPC security group rules, network subnet selection, IAM instance role, and EBS volume configuration.



Why It Matters for Automation

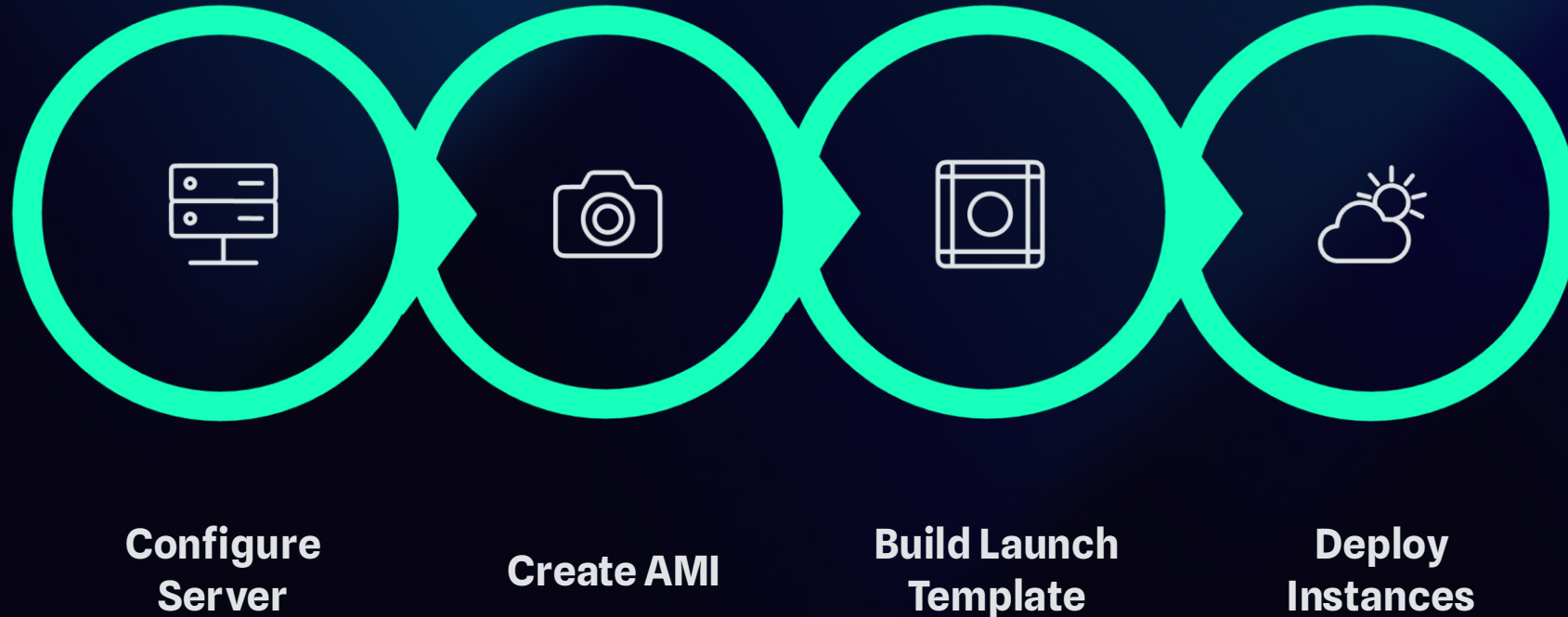
Once saved, a Launch Template is an automated blueprint. Tell AWS to use it to spin up 1 instance or 100 — every single one will be a **carbon copy**, perfectly configured with zero human error.



Infrastructure as Code (IaC)

This is the core principle of modern cloud operations. Instead of clicking through a console wizard manually, you define infrastructure in a template and let automation handle the rest.

AMI + Launch Template = Scalability Foundation



This pipeline is the backbone of every production-grade cloud deployment. Once your AMI and Launch Template are in place, AWS Auto Scaling can use them to clone your infrastructure in seconds – without a human touching a keyboard. That's the power of automation in action.

Chapter 4

Auto Scaling Groups

What happens when your app gets more traffic than a single server can handle? Auto Scaling Groups are the cloud's answer to elastic, hands-free capacity management.



Vertical vs. Horizontal Scaling: Understanding the Difference

Vertical Scaling (Scale UP)

Make the existing server **bigger** – upgrade from a t3.large to a t3.xlarge. Adds more vCPUs and RAM to a single machine.

Problems: Requires a reboot (downtime!), and you still have a single point of failure. If that one big server crashes, everything goes offline.

Horizontal Scaling (Scale OUT)

Add more **identical servers** side by side instead of making one bigger. Traffic is shared across all of them. **Advantages:** No downtime. If one server fails, the others keep serving users. This is the cloud-native approach – and what AWS is designed for.



Horizontal scaling is the foundation of every highly available AWS architecture. Design for scale-out, not scale-up.



Auto Scaling Groups (ASG): Hands-Free Capacity Management

An **Auto Scaling Group (ASG)** is a logical boundary around a fleet of EC2 instances. It monitors your application's performance metrics and automatically adds or removes servers based on real-time demand — all without human intervention.

Min

Minimum Size

The fewest servers that must always run, even during off-peak hours. Keeps costs low during quiet times.

Des

Desired Capacity

Your normal operational baseline. The ASG will always try to maintain this number of healthy instances.

Max

Maximum Size

The ceiling that prevents runaway cloud bills during unexpected traffic spikes. Your cost guardrail.

ASG in Action: A Real-World Example

Let's walk through how an ASG responds to a traffic surge at an e-commerce company during a flash sale event.

8:00 AM — Normal Traffic

ASG maintains **2 instances** (Desired capacity).
CPU usage sits at 30%. All quiet.

12:02 PM — ASG Scales Out

Using the Launch Template, **3 new identical instances** launch automatically. Load distributes across all 5 servers. CPU drops back to normal.

1

2

3

4

12:00 PM — Flash Sale Begins

Traffic surges. CPU across the fleet climbs above **70%**. CloudWatch alarm triggers the ASG scale-out policy.

6:00 PM — Sale Ends

Traffic subsides. CPU drops below 30%. ASG safely **terminates 3 extra instances** to save money. Back to 2 running.

Configuring an ASG Scaling Policy

Your ASG needs a **scaling policy** – a rule that tells it when to add or remove instances. The most common type is a **Target Tracking Scaling Policy**.

Scale-Out Trigger (Add Servers)

Example rule: *"If average CPU utilization across all instances in the ASG exceeds **70%** for 2 consecutive minutes, launch 2 additional EC2 instances using the saved Launch Template."*

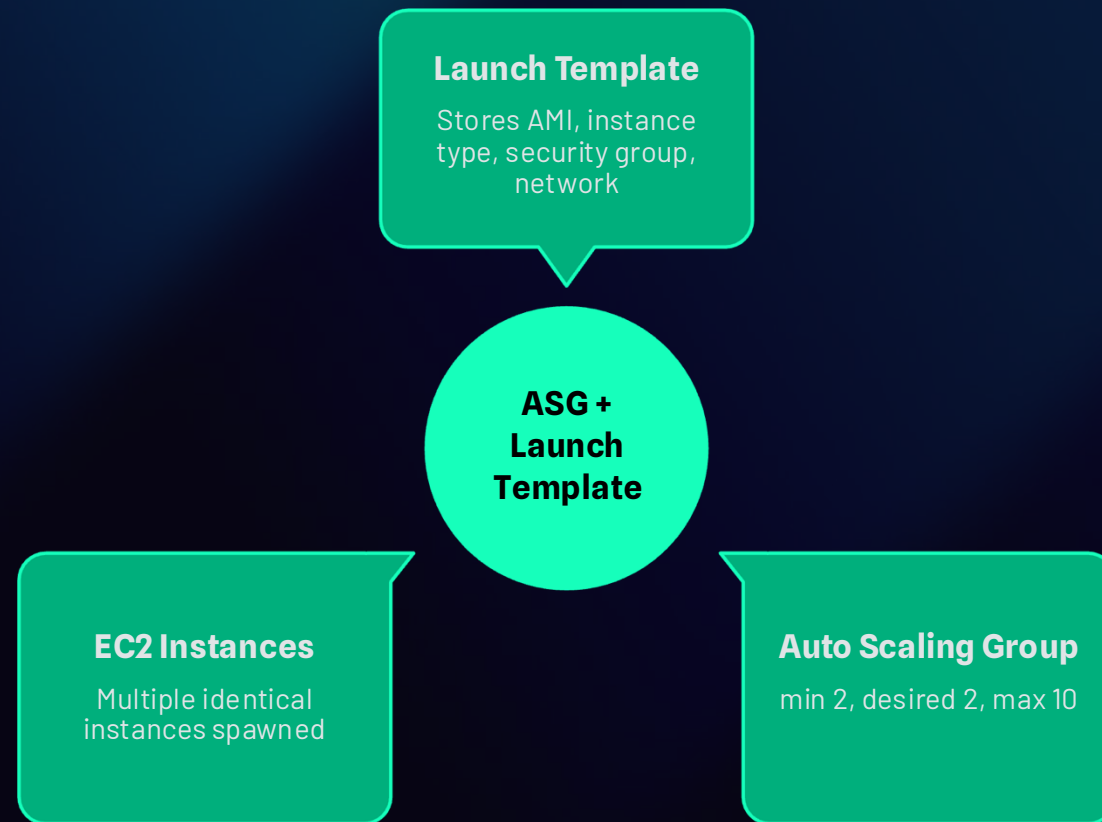
CloudWatch monitors the metric. When the alarm fires, the ASG acts immediately – no human needed.

Scale-In Trigger (Remove Servers)

Example rule: *"If average CPU utilization drops below **30%** for 5 consecutive minutes, terminate 1 instance from the fleet."*

ASG always terminates the instance in the Availability Zone with the most instances first, keeping your fleet balanced across zones.

ASG + Launch Template: The Automation Pair



The **Launch Template** is the blueprint. The **Auto Scaling Group** is the construction manager. Together they form a self-healing, elastic infrastructure layer that can respond to business demand in real time — the hallmark of modern cloud architecture.

- ❑ ASG also provides **self-healing**: if an instance fails a health check and becomes unhealthy, the ASG automatically terminates it and launches a replacement using the Launch Template.

The New Problem: Multiple Servers, One URL

The Challenge 🙄

Auto Scaling solved our capacity problem. But now we've created a new one. If our ASG is running **5 separate EC2 instances** to handle traffic, which server should a user's browser connect to?

Each instance has its own unique IP address. If we publish 5 different IP addresses in our DNS, users would be randomly distributed – but there's no intelligence, no health checking, and no guarantee the server they hit is actually healthy.

The Solution

We need a single, intelligent entry point that:

- Accepts all incoming traffic at one URL
- Checks which servers are healthy
- Distributes requests evenly and intelligently
- Spans multiple Availability Zones

That's exactly what an **Elastic Load Balancer** does.



Chapter 5

Elastic Load Balancing

The final piece of a production-grade AWS architecture. An Elastic Load Balancer is the intelligent traffic manager that makes your application highly available and resilient.

What Is Elastic Load Balancing (ELB)?

An **Elastic Load Balancer (ELB)** acts as a single, highly available public entry point for your web application. It sits in front of your fleet of EC2 instances and distributes every incoming user request intelligently across healthy servers.



Single Public Entry Point

All users connect to one DNS name (like `myapp.us-east-1.elb.amazonaws.com`). They never know — or care — how many servers are behind it.



Automated Health Checks

The ELB continuously pings each backend instance. If a server fails to respond, the ELB immediately stops sending it traffic — keeping users on healthy servers only.



Even Traffic Distribution

Requests are spread evenly across all healthy instances. No single server gets overwhelmed while others sit idle — perfect load distribution.

Types of AWS Load Balancers

Application Load Balancer (ALB) — Layer 7

The most commonly used type. Operates at the **HTTP/HTTPS application layer**. Can route traffic based on URL path (`/api` vs. `/images`), hostname, or HTTP headers. The best choice for web applications and microservices.

Network Load Balancer (NLB) — Layer 4

Operates at the **TCP/UDP transport layer**. Handles millions of requests per second with ultra-low latency. Used for gaming servers, IoT, financial trading platforms where milliseconds matter.

Gateway Load Balancer (GWLB)

Used to deploy, scale, and manage **third-party virtual network appliances** — like firewalls, intrusion detection systems, and deep packet inspection tools. Specialized enterprise use case.



For the AWS Cloud Practitioner exam and most job interviews, focus on the **Application Load Balancer (ALB)** — it's the most tested and most commonly deployed.

Application Load Balancer (ALB)

Deep Dive

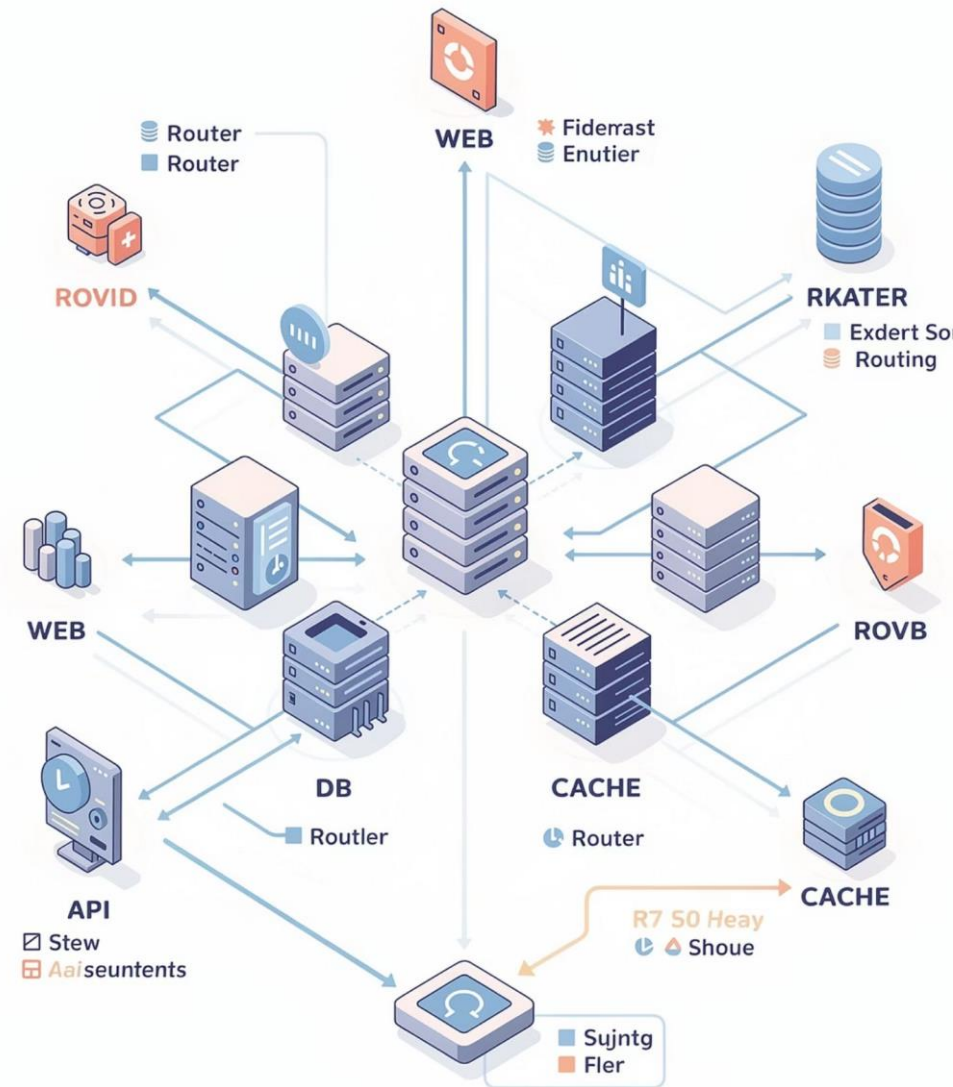
The ALB operates at **Layer 7 of the OSI model** — the application layer. This means it can read and understand HTTP/HTTPS request content, not just TCP packets, giving it powerful intelligent routing capabilities.

Path-Based Routing

Route `/api/*` requests to your backend API server fleet, and `/images/*` requests to your static asset servers — all from one load balancer.

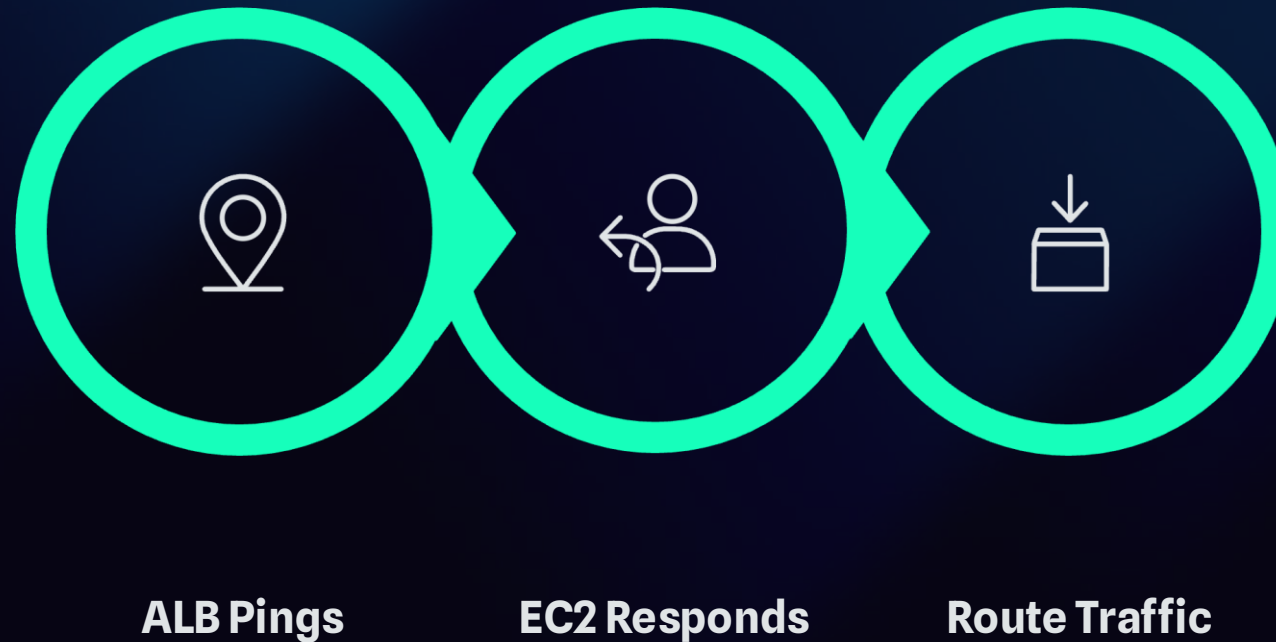
Host-Based Routing

Route `shop.mycompany.com` to one target group and `admin.mycompany.com` to a completely separate, more locked-down fleet.



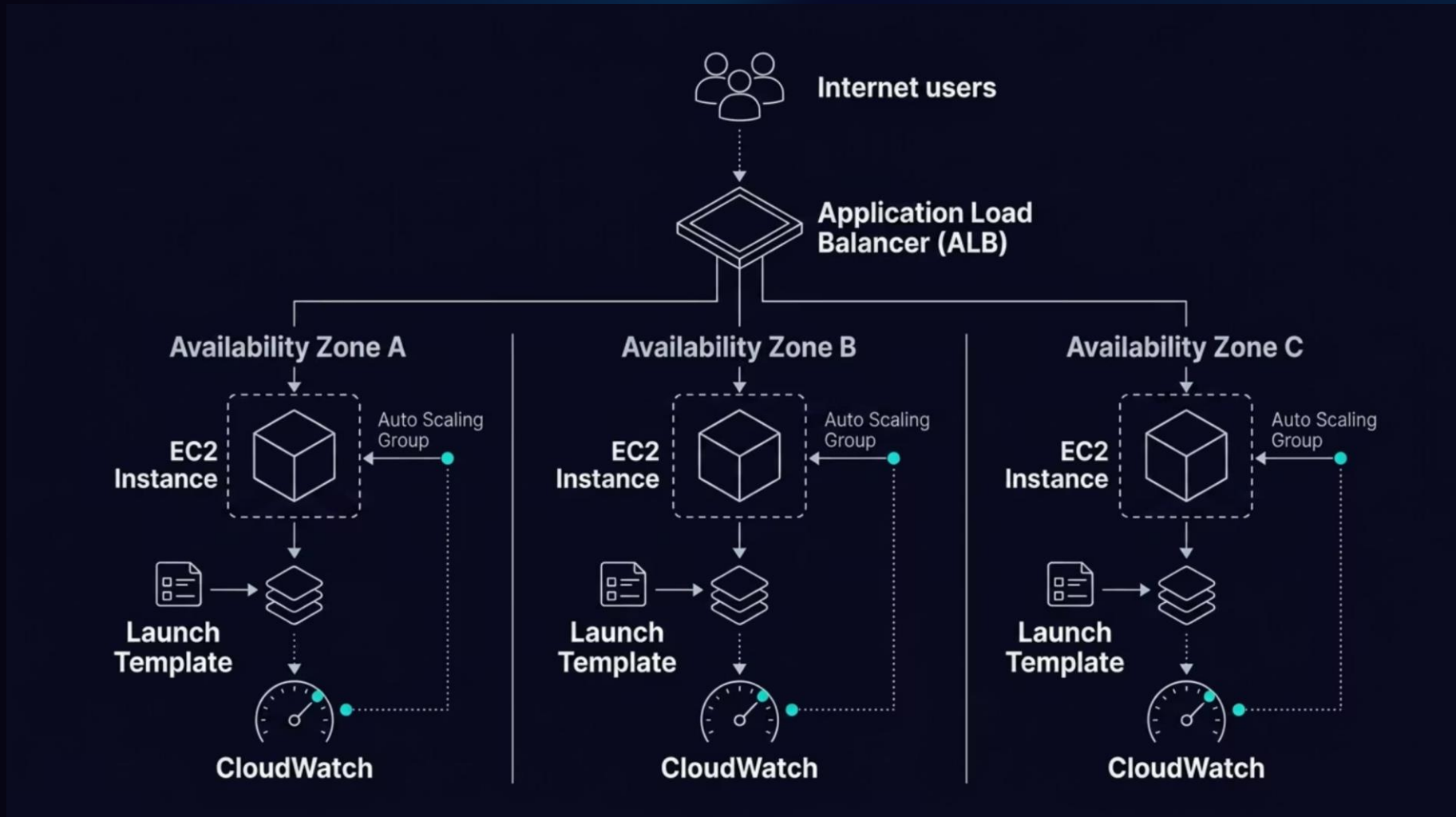
How ALB Health Checks Work

Health checks are one of the most important features of the Application Load Balancer. They protect your users from ever hitting a broken server.



The ALB pings each registered EC2 instance at a configured interval (commonly every 30 seconds). If an instance returns a **healthy response (HTTP 200)**, traffic continues. If it fails to respond or returns an error, the ALB immediately pulls that instance out of rotation – users never notice a thing. This is what makes your application **self-healing** at the network level.

The Complete Architecture: ELB + ASG Together



Why Availability Zones Matter for Load Balancing

What Are Availability Zones?

AWS Regions (like US-East-1) contain multiple **Availability Zones (AZs)** — physically separate data centers with independent power, cooling, and networking. If one data center has a fire or power failure, the others keep running.

When you configure an ALB, you **enable it across multiple AZs**. It distributes your EC2 instances across these zones so that even a full data center outage only affects a portion of your fleet.

Real-World Impact

Without multi-AZ architecture:

- One data center failure = full app outage
- Maintenance window = downtime for all users
- No geographic redundancy

With multi-AZ + ALB:

- One AZ fails → ALB routes to other AZs instantly
- Zero downtime for end users
- True high availability architecture 

Putting It All Together: Architecture Design Lab

Let's design a complete, production-ready web application architecture using everything we've learned today. This is the kind of design exercise you'll face in a cloud support interview.

1

Step 1: AMI + Launch Template

Configure your web server, install your app, bake it into an AMI, and save all settings in a Launch Template.

2

Step 2: Auto Scaling Group

Create an ASG using the Launch Template. Set Min=2, Desired=2, Max=10. Configure a CloudWatch CPU alarm at 70% to trigger scale-out.

3

Step 3: Application Load Balancer

Deploy an ALB across 3 Availability Zones. Register the ASG instances as a Target Group. Configure health checks every 30 seconds.

4

Step 4: Route 53 DNS

Point your domain name to the ALB's DNS endpoint. Users now hit a single URL backed by an elastic, self-healing, multi-AZ infrastructure.

Key Vocabulary Checkpoint

Before we move to the hands-on lab, let's make sure you're solid on the essential terms. These appear on the AWS Cloud Practitioner exam and in every cloud support job interview.

Term	Definition
EC2	Elastic Compute Cloud — AWS's virtual machine service. Rent compute power by the second.
EBS	Elastic Block Store — A network-attached persistent storage volume for EC2 instances.
AMI	Amazon Machine Image — A frozen snapshot of an OS + software configuration used to clone instances.
Launch Template	A saved configuration blueprint specifying instance type, AMI, security groups, and network settings.
ASG	Auto Scaling Group — Automatically adds or removes EC2 instances based on demand metrics.
ELB / ALB	Elastic Load Balancer / Application Load Balancer — Distributes web traffic across healthy EC2 instances.

Common Exam Scenarios: Test Your Knowledge

These are the types of questions that appear on the AWS Cloud Practitioner exam. Work through each one mentally before reading the answer.

1 A company needs to run a nightly data processing job that can tolerate interruptions. Which pricing model minimizes cost?

Answer: Spot Instances. The 2-minute termination warning is acceptable for batch jobs that can checkpoint and restart.

2 An app's response time degrades during peak hours but recovers overnight. Which AWS feature automatically resolves this?

Answer: Auto Scaling Group with a Target Tracking Scaling Policy tied to CPU utilization or request count metrics.

3 A web app must remain available even if an entire AWS data center loses power. What architecture ensures this?

Answer: ALB + ASG deployed across multiple Availability Zones. Traffic automatically reroutes to healthy AZs.

Learning Objectives: Did We Hit Our Targets?

Let's revisit today's Bloom's Taxonomy-aligned objectives and confirm what you're now able to do.



Cognitive / Analysis

You can now select the most cost-efficient EC2 pricing model (On-Demand, Spot, Reserved, Savings Plans) based on workload characteristics like runtime predictability and interruption tolerance.



Psychomotor / Application

You can create an Auto Scaling policy blueprint — defining Min/Desired/Max sizes and linking a CloudWatch metric alarm to dynamically adjust capacity with demand.



Psychomotor / Creation

You can configure an ALB target group infrastructure — registering EC2 instances, enabling multi-AZ distribution, and implementing health checks for a production-ready web application.

Hands-On Lab: Build It Yourself

The best way to lock in today's concepts is to build the architecture yourself. Your lab environment is ready.



Launch an EC2 Instance

Use the AWS Console to launch a `t3.micro` Amazon Linux 2 instance. Explore the instance type selector and pricing options.



Build a Launch Template

Create a Launch Template using your custom AMI. Configure the instance type, security group (allow port 80), and network settings.



Create a Custom AMI

Install a simple web server on your running instance, then create an AMI snapshot of it. Save it as your deployment image.



Configure ASG + ALB

Create an Auto Scaling Group using your Launch Template (Min=1, Desired=2, Max=4). Attach an Application Load Balancer and verify traffic distribution.

Day 5 Key Takeaways

You covered serious ground today. Here's everything you've added to your cloud technician toolkit.

EC2 Fundamentals

Virtual machines with isolated vCPU, RAM, and separate EBS storage. Instance families (T/M, C, R) match compute, memory, and general workloads respectively.

Cost Optimization

On-Demand = flexibility.
Reserved/Savings Plans = up to 72% savings for steady workloads. Spot = up to 90% savings for interruptible batch jobs.

AMI + Launch Templates

The automation blueprint. AMI freezes your server image. Launch Templates package all deployment settings for instant, error-free cloning.

Auto Scaling Groups

Horizontal scaling that automatically responds to demand. Min/Desired/Max parameters. CloudWatch alarms trigger scale-out and scale-in policies.

Elastic Load Balancing

ALB distributes HTTP/HTTPS traffic across healthy EC2 instances across multiple Availability Zones. Health checks ensure users only hit working servers.

What's Coming on Day 6

You've mastered compute. Next session, we move into the storage and database layer — the persistent data foundation that powers everything your EC2 instances process.

Amazon S3

Object storage for files, backups, static websites, and data lakes. Virtually unlimited, 99.99999999% durability.

Amazon RDS

Managed relational databases (MySQL, PostgreSQL, SQL Server). Automated backups, multi-AZ replication, and read replicas.

Amazon DynamoDB

Serverless NoSQL database. Single-digit millisecond performance at any scale. Perfect for mobile apps and gaming backends.

- Review today's lab before next class. Being able to explain your ASG + ALB setup in plain English is excellent interview preparation.





You're Building Real Cloud Skills

Keep Going

The architecture patterns you learned today – EC2, Auto Scaling, and Load Balancing – appear in nearly **every AWS job description** at every experience level. You're not just learning theory. You're building the practical vocabulary and hands-on muscle memory that employers are actively hiring for.

"The best cloud technicians aren't the ones who memorized the most services – they're the ones who understand **why** each piece of the architecture exists and **when** to use it."

See you next session. Keep building. 🚀